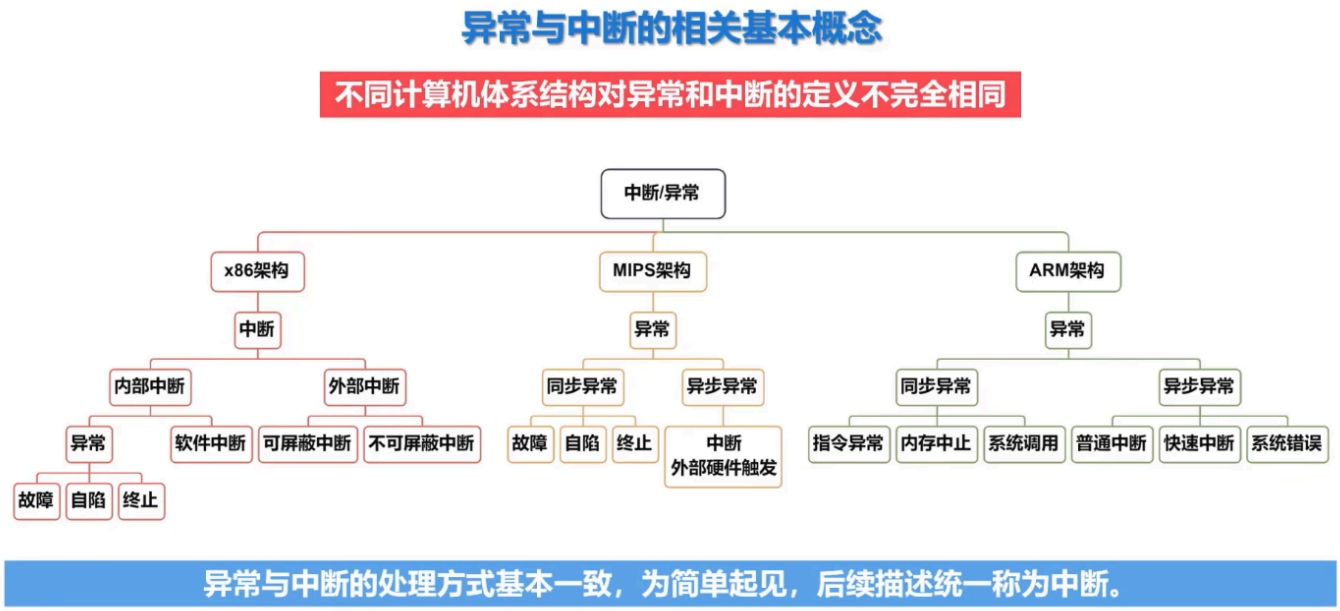
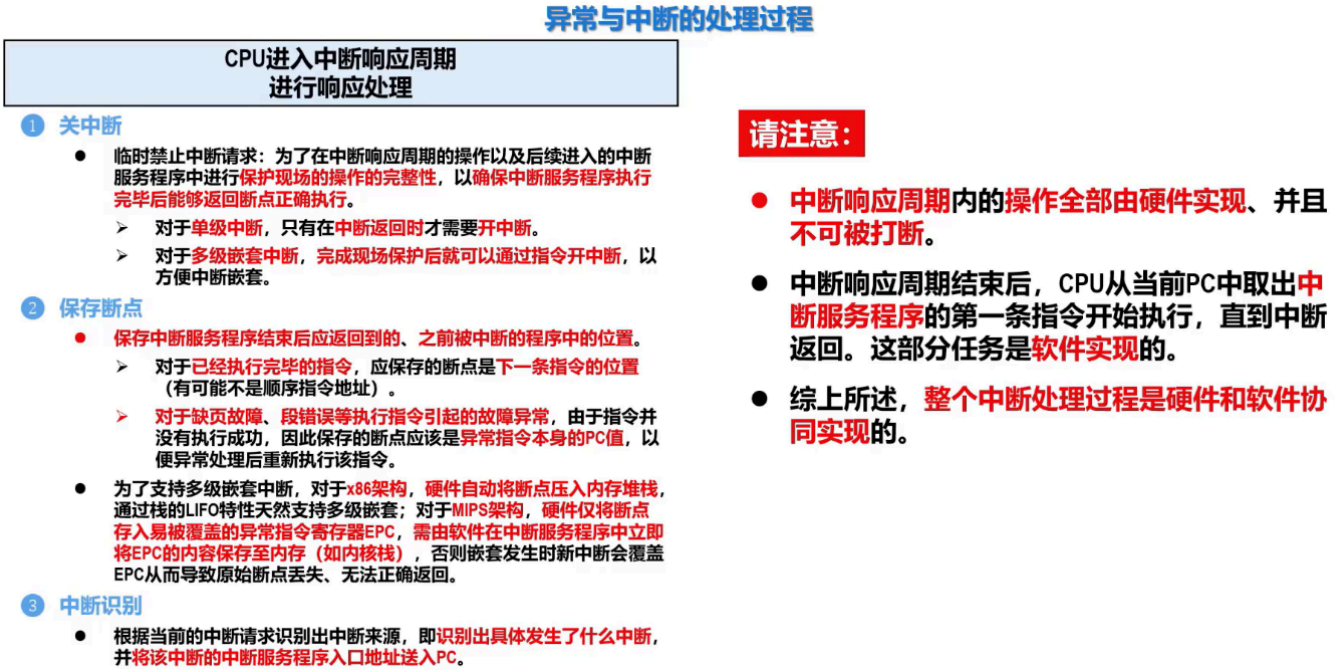
不同架构对异常和中断的命名有差异：





一、总体框架：硬件与软件的接力

整个处理过程以**中断响应周期结束**为分界线，分为两个阶段，这正是"硬件与软件协同实现"的具体含义：

- **硬件阶段（中断隐指令）**：CPU响应中断后进入**中断响应周期**，硬件自动完成三件事——关中断、保存断点、中断识别，这一阶段全部由硬件实现且**不可被打断**，没有对应任何机器指令，程序员无法干预
- **软件阶段（中断服务程序ISR）**：响应周期结束后，CPU从当前PC取出ISR第一条指令开始执行，直到中断返回，这部分全部由软件（操作系统）实现

分界的原因很清晰：保存断点只需处理PC一个值，硬件做最快且必须抢先做；保护现场要保存哪些寄存器取决于ISR代码用什么寄存器，硬件无法预知，只能交给软件

二、硬件阶段：中断响应周期的三步操作

第1步 关中断

硬件自动置中断允许标志为0（如x86的IF），临时禁止新的中断请求
目的：保证响应周期内的操作以及后续保护现场操作的**完整性**——若保存断点或现场的中途被新中断打断，现场会被破坏，服务程序执行完毕后无法正确返回断点
开中断的时机因嵌套策略而异：
- **单级中断**：整个ISR执行期间保持关中断，直到**中断返回时**才开中断
- **多级嵌套中断**：完成断点与现场保护后即可通过指令**开中断**（配合屏蔽字），允许更高优先级中断进入嵌套

第2步 保存断点

断点 = ISR结束后应返回的、被打断程序中的位置，**断点指向哪条指令取决于事件类型**、

事件类型	指令执行状态	保存的断点	返回后行为
外部中断、自陷Trap	指令已执行完毕	**下一条指令**的地址（可能不是顺序地址）	顺序继续
缺页、段错误等Fault	指令未执行成功	**异常指令本身的PC值**	修复后重新执行该指令
终止Abort	硬件严重故障	无意义	终止进程
两个重要细节：			
细节一：断点"可能不是顺序指令地址"——若被打断的指令是转移指令且已执行完，PC中装的已是转移目标地址，保存的就是这个目标地址，这从侧面印证了PC在取指阶段自增、执行阶段可被覆盖的设计逻辑			
细节二：断点存放位置因架构而异，直接决定嵌套支持方式：			
- **x86**：硬件自动将断点**压入内存堆栈**，利用栈的LIFO特性天然支持多级嵌套，返回时逐层弹出			
- **MIPS**：硬件仅将断点存入**专用寄存器EPC**，单值寄存器会被新中断覆盖，因此软件必须在ISR开头**立即将EPC内容保存至内核栈**，否则嵌套中断会覆盖EPC，导致原始断点丢失、无法正确返回			
### 第3步 中断识别			
根据当前中断请求识别出**中断来源**（具体发生了什么事件），并将对应ISR的**入口地址送入PC**，CPU由此跳转执行			
识别方式两大类：			
- **向量中断（硬件识别）**：中断源给出中断类型号，CPU以类型号为索引查**中断向量表**得到入口地址，速度快，为主流方案			
- **软件查询（软件识别）**：硬件只跳到固定查询程序入口，由软件逐个询问设备确定中断源，灵活但慢			
## 三、软件阶段：中断服务程序的执行			
响应周期结束后，CPU从当前PC（已指向ISR入口）取指执行，典型流程：			
1. **保护现场**：将ISR会用到的通用寄存器内容压栈			
2. **开中断**（若支持嵌套）：设置屏蔽字，放行更高优先级中断			
3. **中断处理**：执行具体服务逻辑（读键盘缓冲、搬移数据等）			
4. **关中断**：防止恢复现场的过程被打断			

事件类型	指令执行状态	保存的断点	返回后行为
5. **恢复现场**：从栈中弹回寄存器内容			
6. **中断返回**：执行IRET（x86）/ eret（MIPS），恢复断点与程序状态字，回到被打断的程序			
若支持嵌套，步骤3期间到来的更高优先级中断会再次触发一套完整的"硬件响应周期 + 软件保存现场"，形成**后进先出的现场栈恢复链**			
## 四、关键考点提炼			
1. **软硬件分界**：响应周期内三步（关中断、保存断点、中断识别）全由硬件完成且不可打断；保护现场、恢复现场、中断处理全由软件完成——真题最爱混淆"保护现场"的归属			
2. **断点位置分类**：Fault存**当前指令**PC以便重执行（缺页异常是命题热点）；Trap与中断存**下一条指令**地址			
3. **开中断时机**：单级在返回时才开，嵌套在保护现场后即开			
4. **断点保存机制对比**：x86硬件压栈天然支持嵌套，MIPS单靠EPC会被覆盖、必须软件及时保存			
5. **响应三条件**：CPU开中断（IF=1）、请求未被屏蔽、当前指令已执行完毕，三者缺一不可			
6. **中断向量表**：类型号到ISR入口地址的映射结构，位于内存中，向量中断的核心			
7.			
一张图记住主线：**指令边界 → 硬件三连（关中断、存断点、送入口）→ 软件接力（存现场、处理、恢复、返回）**，硬件负责"保住回程的地址"，软件负责"保住执行的环境"，二者缺一中断机制即失效			